

CS214 Written Assignment HW2

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 5/11/2026

Introduction

In this written assignment, I solve several problems related to parallel algorithms. The main topics include computing Fibonacci numbers with parallel prefix, designing and analyzing parallel merging algorithms, creating a parallel friend-recommendation algorithm, and proving a high-probability retry bound for hash-table insertion. For each problem, I explain the main algorithm idea and then justify the work and span bounds.

1 Fibonacci Numbers

We are given the Fibonacci recurrence

$$F_0 = 0, \quad F_1 = 1, \quad F_i = F_{i-1} + F_{i-2}.$$

The vector is defined as

$$v_i = \begin{bmatrix} F_{i-1} \\ F_i \end{bmatrix}.$$

Question 1: Transition Matrix

We want to find a 2×2 matrix A such that

$$Av_i = v_{i+1}.$$

Since

$$v_i = \begin{bmatrix} F_{i-1} \\ F_i \end{bmatrix}, \quad v_{i+1} = \begin{bmatrix} F_i \\ F_{i+1} \end{bmatrix} = \begin{bmatrix} F_i \\ F_{i-1} + F_i \end{bmatrix},$$

we need a matrix whose first row picks out the second component, and whose second row adds both components. Therefore,

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Checking the multiplication,

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_{i-1} \\ F_i \end{bmatrix} = \begin{bmatrix} F_i \\ F_{i-1} + F_i \end{bmatrix} = \begin{bmatrix} F_i \\ F_{i+1} \end{bmatrix} = v_{i+1}.$$

So the answer is

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Question 2: Parallel Prefix Algorithm

From Question 1, we know that

$$v_{i+1} = Av_i.$$

Therefore,

$$v_i = A^{i-1}v_1,$$

where

$$v_1 = \begin{bmatrix} F_0 \\ F_1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

So to compute $F_1, F_2, \dots, F_n$, it is enough to compute

$$A^0, A^1, A^2, \dots, A^{n-1}.$$

The value A^0 is just the identity matrix, so v_1 is already known. For the remaining values, we can run a parallel prefix product on the sequence

$$A, A, A, \dots, A$$

with $n - 1$ copies of A . The prefix products give

$$A^1, A^2, A^3, \dots, A^{n-1}.$$

Then for each i , we compute

$$v_i = A^{i-1}v_1.$$

The second component of v_i gives the desired Fibonacci number F_i .

Since each matrix is only 2×2 , multiplying two matrices takes constant work. Matrix multiplication is also associative, so it can be used in a parallel prefix scan. A parallel prefix scan on n elements takes

$$O(n)$$

work and

$$O(\log n)$$

span. After the scan, multiplying each matrix power by v_1 also takes $O(n)$ total work and $O(1)$ additional span if done in parallel.

Therefore, the total cost is

$$\boxed{O(n) \text{ work and } O(\log n) \text{ span.}}$$

2 Parallel Merging

Assume that A and B are two sorted arrays, and all keys are distinct. Let

$$n = |A| + |B|.$$

Question 1: Select the k -th Element

We are given two sorted arrays A and B , and we want to find the k -th smallest element in $A \cup B$ using

$$O(\log(|A| + |B|))$$

work.

The handout uses

$$0 \leq k < |A| + |B|,$$

so I treat k as a zero-indexed rank. Equivalently, I want the element with one-indexed rank

$$r = k + 1.$$

The main idea is to binary search how many elements should be taken from A . Suppose we take i elements from A and j elements from B , where

$$i + j = r.$$

To make sure both i and j are valid, we binary search over

$$\max(0, r - |B|) \leq i \leq \min(r, |A|).$$

For boundary cases, we can treat missing values as sentinels:

$$A[-1] = B[-1] = -\infty, \quad A[|A|] = B[|B|] = +\infty.$$

Then the cut is correct if

$$A[i - 1] \leq B[j]$$

and

$$B[j - 1] \leq A[i].$$

These two conditions mean that every element on the left side of the cut is smaller than or equal to every element on the right side of the cut.

The algorithm is:

1. Binary search on i , the number of elements taken from A .
2. Let $j = r - i$.
3. If $A[i - 1] > B[j]$, then we took too many elements from A , so move i left.
4. If $B[j - 1] > A[i]$, then we took too few elements from A , so move i right.
5. Otherwise, the correct cut has been found.

At the end, the k -th element is

$$\max(A[i - 1], B[j - 1]).$$

The algorithm only does a binary search over one valid range of indices. Therefore, the work is

$$O(\log(\min(|A|, |B|))) \leq O(\log(|A| + |B|)).$$

So the required work bound is

$$\boxed{O(\log(|A| + |B|))}.$$

Question 2: Work and Span for Idea 1

In Idea 1, the algorithm first chooses pivots at the global ranks

$$k, 2k, 3k, \dots, mk,$$

where

$$m = \left\lfloor \frac{n}{k} \right\rfloor.$$

Each pivot can be found by using the select k -th element subroutine from Question 1. Since each select operation costs

$$O(\log n)$$

work, and there are about

$$\frac{n}{k}$$

pivots, the total work for finding all pivots is

$$O\left(\frac{n}{k} \log n\right).$$

After the pivots are found, they split A and B into slices. Each slice pair has total size at most about k . In Idea 1, each slice pair is merged sequentially. Since the total size of all slice pairs is n , the total work for all sequential merges is

$$O(n).$$

Therefore, the total work is

$$W(n, k) = O\left(n + \frac{n}{k} \log n\right).$$

For span, the pivot selections can be done in parallel, so the span for finding pivots is

$$O(\log n).$$

Then all slice pairs are merged in parallel. However, each slice pair is merged sequentially, and the largest slice has size at most about k . Therefore, the span of the merging step is

$$O(k).$$

So the total span is

$$S(n, k) = O(\log n + k).$$

Question 3: Choosing k for Idea 1

We want the algorithm to be work-efficient, meaning the total work should be

$$O(n).$$

From Question 2, the work is

$$W(n, k) = O\left(n + \frac{n}{k} \log n\right).$$

To make this $O(n)$, we need

$$\frac{n}{k} \log n \leq O(n).$$

Dividing both sides by n , this means

$$\frac{\log n}{k} \leq O(1).$$

So we can choose

$$k = \Theta(\log n).$$

With this choice,

$$W(n, k) = O\left(n + \frac{n}{\log n} \log n\right) = O(n + n) = O(n).$$

The span from Question 2 is

$$S(n, k) = O(\log n + k).$$

Substituting

$$k = \Theta(\log n),$$

we get

$$S(n, k) = O(\log n + \log n) = O(\log n).$$

Therefore, choosing

$$k = \Theta(\log n)$$

gives

$$O(n) \text{ work and } O(\log n) \text{ span.}$$

Question 4: Work and Span Recurrences for Idea 2

In Idea 2, the algorithm again chooses pivots at the global ranks

$$k, 2k, 3k, \dots, mk,$$

where approximately

$$m = \frac{n}{k}.$$

Each pivot is found using the select algorithm from Question 1, which costs

$$O(\log n)$$

work. Therefore, the total work for finding pivots at one level is

$$O\left(\frac{n}{k} \log n\right).$$

The pivots split the input into smaller subproblems. Each subproblem has size at most about k , and there are about

$$\frac{n}{k}$$

subproblems. Since Idea 2 recursively merges each slice pair, the work recurrence is

$$W(n) = \frac{n}{k}W(k) + O\left(\frac{n}{k} \log n\right).$$

For span, all pivot selections can be done in parallel. Thus, the span for selecting pivots is

$$O(\log n).$$

After the pivots are found, all recursive subproblems run in parallel, so the span only follows the largest subproblem. Since each subproblem has size at most about k , the span recurrence is

$$S(n) = S(k) + O(\log n).$$

Question 5: Span Bound for Idea 2

Now choose

$$k = \Theta(\sqrt{n}).$$

The span recurrence from Question 4 becomes

$$S(n) = S(\sqrt{n}) + O(\log n).$$

Expanding this recurrence,

$$S(n) = O(\log n) + O(\log \sqrt{n}) + O(\log n^{1/4}) + O(\log n^{1/8}) + \dots.$$

Since

$$\log \sqrt{n} = \frac{1}{2} \log n,$$

$$\log n^{1/4} = \frac{1}{4} \log n,$$

and so on, this becomes

$$S(n) = O\left(\log n + \frac{1}{2} \log n + \frac{1}{4} \log n + \dots\right).$$

The terms form a geometric series, so the sum is bounded by a constant multiple of $\log n$. Therefore,

$$S(n) = O(\log n).$$

Since

$$n = |A| + |B|,$$

the span is

$$\boxed{O(\log(|A| + |B|))}.$$

Question 6: Work Bound for Idea 2

Again choose

$$k = \Theta(\sqrt{n}).$$

The work recurrence from Question 4 becomes

$$W(n) = \sqrt{n}W(\sqrt{n}) + O(\sqrt{n} \log n).$$

Now divide both sides by n :

$$\frac{W(n)}{n} = \frac{W(\sqrt{n})}{\sqrt{n}} + O\left(\frac{\log n}{\sqrt{n}}\right).$$

Let

$$T(n) = \frac{W(n)}{n}.$$

Then

$$T(n) = T(\sqrt{n}) + O\left(\frac{\log n}{\sqrt{n}}\right).$$

Expanding the recurrence gives

$$T(n) = T(\sqrt{n}) + O\left(\frac{\log n}{\sqrt{n}}\right),$$

$$T(\sqrt{n}) = T(n^{1/4}) + O\left(\frac{\log \sqrt{n}}{n^{1/4}}\right),$$

and so on. Therefore,

$$T(n) = T(O(1)) + O\left(\frac{\log n}{\sqrt{n}}\right) + O\left(\frac{\frac{1}{2} \log n}{n^{1/4}}\right) + O\left(\frac{\frac{1}{4} \log n}{n^{1/8}}\right) + \dots.$$

This sum is bounded by a constant: the early terms are small because the denominator is large, while the later terms are also controlled because the logarithmic factor has already decreased many times through repeated square roots. Therefore,

$$T(n) = O(1).$$

Since

$$T(n) = \frac{W(n)}{n},$$

we get

$$W(n) = O(n).$$

Thus, when

$$k = \Theta(\sqrt{n}),$$

Idea 2 has

$O(n)$ work.

3 Friend Recommendation

We are given a user u , and we want to return all of u 's friends' friends. However, we should not include u itself, and we should not include users who are already direct friends of u .

The available APIs are:

`get_friends(user u)`

which returns the array of friend IDs of user u , and

`num_friends(user u)`

which returns the number of friends of user u .

Question 1: Parallel Friend-Recommendation Algorithm

Let

$$F = \text{get_friends}(u).$$

First, I build a set called `directSet`. This set contains u itself and every direct friend of u . This lets me quickly test whether a user should be excluded from the recommendation list.

Next, for every friend $f \in F$, I compute

$$d_f = \text{num_friends}(f).$$

Using a parallel prefix sum over these degrees, I compute offsets for where each friend list should be placed in one flattened array. Then I copy all arrays

`get_friends(f)`

for all $f \in F$ into one large array L . This array L contains all two-hop candidates before filtering.

Then, in parallel, I process every vertex $v \in L$. If

$v \notin \text{directSet},$

then v is a valid candidate recommendation. I insert v into another set called `candidateSet`. Using a set also removes duplicate recommendations, since the same user could appear through multiple mutual friends.

Finally, I output all elements of `candidateSet` as the array R . The size of R is just the number of elements in `candidateSet`.

More Detailed Steps

1. Let $F = \text{get_friends}(u)$.
2. Insert u and all users in F into `directSet`.
3. In parallel, compute $d_f = \text{num_friends}(f)$ for every $f \in F$.
4. Use parallel prefix sum on the values d_f to compute offsets into a flattened array L .
5. In parallel, copy each `get\_friends(f)` into its assigned segment of L .
6. In parallel, for each $v \in L$, if $v \notin \text{directSet}$, insert v into `candidateSet`.
7. Compact the elements of `candidateSet` into the output array R and return its size.

Work Analysis

The algorithm scans the friend list of u once. Then it scans the friend lists of all friends of u .

Since the friendship graph is bi-directional, the total number of entries across all friend lists in the whole graph is $O(m)$. Therefore, the number of entries scanned by this algorithm is at most $O(m)$.

The prefix sums, copying steps, and hash-set insertions/lookups are also linear in the number of processed entries. I am using the sets as sparse parallel hash sets, so I do not need to scan all n users.

Therefore, the total work is

$$\boxed{O(m)}.$$

Span Analysis

The degree computations and copying steps are done in parallel. The main parallel subroutine used is prefix sum, which has span

$$O(\log n).$$

The lookups in `directSet` and insertions into `candidateSet` can also be done in parallel using hash tables. These operations do not create a long sequential chain.

Finally, compacting the output can be done with another parallel prefix sum. Therefore, the overall span is

$$\boxed{O(\log n)}.$$

So the algorithm satisfies the required bounds:

$$\boxed{O(m) \text{ work and } O(\log n) \text{ span.}}$$

4 With High Probability and Hash Table

Question 1: Retry Bound with High Probability

For one insertion into the hash table, let t be the number of retries. The load factor is less than $1/2$, so fewer than half of the slots are already taken.

Each retry chooses a random slot independently. Since fewer than half of the slots are full, the probability that the chosen slot is full is at most

$$\frac{1}{2}.$$

Therefore, the probability of having a collision on one try is at most

$$\frac{1}{2}.$$

The insertion keeps retrying only if it keeps hitting occupied slots. So for any integer r ,

$$\Pr[t \geq r] \leq \left(\frac{1}{2}\right)^r.$$

Now choose

$$r = ck \log n$$

for a large enough constant c . Then

$$\Pr[t \geq ck \log n] \leq \left(\frac{1}{2}\right)^{ck \log n}.$$

Using the fact that

$$\left(\frac{1}{2}\right)^{ck \log n} = n^{-ck \log 2},$$

we can choose a constant c such that

$$c \log 2 \geq 1.$$

Then

$$ck \log 2 \geq k.$$

Therefore,

$$\Pr[t \geq ck \log n] \leq \frac{1}{n^k}.$$

This matches the definition of “with high probability,” using $g(n) = \log n$. Therefore,

$$t = O(\log n)$$

with high probability.

5 Conclusion

In this assignment, I practiced several important techniques for designing parallel algorithms. For Fibonacci numbers, I used a transition matrix and parallel prefix to compute many values efficiently. For parallel merging, I analyzed how blocking, pivot selection, and recursive merging affect both work and span. For friend recommendation, I used parallel flattening, filtering, and compaction to build a recommendation list within the required bounds. Finally, for the hash-table problem, I used a simple probability bound to show that the number of retries is logarithmic with high probability.

Overall, the main idea I learned is that parallel algorithms often need two goals at the same time: keeping the total work close to the best sequential algorithm, while also reducing the span by splitting the problem into independent pieces. Prefix sum, divide-and-conquer, and high-probability analysis are useful tools for achieving these goals.

References

- Professor Yihan Sun, *Parallel Sorting*, CS214 Lecture Slides.
- Professor Yihan Sun, *Parallel Merge sort+Race and concurrency*, CS214 Lecture Slides.
- Professor Yihan Sun, *Race and concurrency*, CS214 Lecture Slides.
- OpenAI. *ChatGPT (GPT-5.3)*. Used for suggesting alternative parallel optimization strategies and provided pseudo-code idea.
https://chatgpt.com/s/t_6a013f2a5ecc819187803ae7dba831a9